StudyMate: AI Study and Learning Agent

EECS3311 Software Design — Stage 1 Design Report

Elena Bahrami Hezaveh
Student Number: **221487616**
October 3, 2026

Design proposal, version 1.1. This document specifies the intended system. Implementation and testing are planned, not completed.

1. Project overview

Students often have lecture notes, limited study time and quiz results in separate places. StudyMate turns their own course materials into source-linked learning activities and a feasible study schedule. Its target users are university students preparing for exams and reviewing difficult topics. The agent retrieves evidence, chooses permitted tools, generates learning content, reads persisted learning history and proposes schedule changes. It is appropriate because interpreting study requests and course text requires flexible language processing, while scoring, time allocation and persistence require predictable software rules.

Scope and technology decisions

Proposed implementation: Java desktop application with Swing GUI, a Java command-line entry point, and local SQLite storage accessed through a repository. PDF/TXT text extraction is isolated behind TextExtractor. One local user and one active course workspace are supported per request. Course creation/selection is supporting functionality, not one of the ten counted features. Planned model: OpenAI GPT-4.1 mini, configured by model ID behind OpenAIAdapter. The model is used for grounded generation and structured tool decisions. API credentials are read from an environment variable and never committed. Availability and account access must be confirmed during Stage 2; changing the provider requires an adapter/configuration change and a documented design revision. Out of scope: OCR, web research, LMS integration, live calendar services, collaborative accounts, essay grading and autonomous external actions. These exclusions keep the ten substantive features feasible for a course project.

Report guide. Sections 1–3 describe scope, architecture and interfaces. Section 4 specifies F01–F10/UC01–UC10. Sections 5–7 show class views, patterns and use cases; Section 8 contains SD01–SD11; Sections 9–10 provide traceability and next steps.

2. Architecture and agent behaviour

Presentation: StudyGUI and StudyCLI translate user input into the same typed Request. Application: AgentController provides the Facade and dispatches to MaterialService, QuizService, ProgressTracker or LearningAgent. Domain: StudyPlanner, StudyPlan, Quiz and learning records express the actual study rules. Infrastructure: StudyRepository, TextExtractor, tool implementations and OpenAIAdapter provide storage, extraction and model access.

Bounded agent loop

LearningAgent.run(request) loads course-scoped context, builds an allowlisted tool schema and calls LLMClient.generate(). A model response is either a typed tool request or a final candidate. The agent validates tool name, course ownership and argument types before ToolRegistry.execute(). It appends tool results and calls the model again. At most six tool/model decision rounds are permitted per request. Exhaustion returns an incomplete-task error rather than an unsupported final answer. Available tools are retrieve (RetrievalTool: selected chunks or keyword-ranked search), progress (ProgressTool: topic results), and planContext (PlanContextTool: existing plan and completed sessions). Tools are read-only. Model-directed arbitrary file access, SQL execution and shell commands are not supported. The application alone commits validated results or confirmed plan changes. Retrieval is deterministic keyword ranking over page-labelled chunks, not an unimplemented promise of semantic search. Requests are scoped to a course and optionally material IDs. A first search with insufficient evidence may be refined within the step budget. No matching evidence yields an explicit insufficiency response.

Validation and failure policy

Uploaded text is evidence, not system instructions. Only supplied source IDs may be cited. Validation checks IDs, schemas, quiz keys, counts and schedule constraints; valid references do not prove every generated claim is correct. Generated content is labelled AI-generated and remains reviewable. On malformed output, one repair attempt is allowed within the budget. Network calls have a planned 30-second timeout and one transient-error retry; authentication errors are reported without retry. SQLite transactions prevent partial imports and duplicated quiz attempts. Plan writes use expected-version checks. A failed model call never overwrites an existing artifact or plan. Context uses selected chunks and bounded recent history; raw API secrets and full document text are excluded from diagnostic logs.

Data and acceptance rules

A study session has a topic, start time, duration and completion status. A valid plan ends before the exam, has positive durations, has no overlapping sessions and fits the supplied dated availability. Completed sessions remain unchanged during adaptation. All dates use a user-selected timezone. Quiz scoring uses stored answer keys; the LLM cannot assign or change scores.

3. GUI and CLI interaction contract

StudyGUI provides Materials, Ask, Plan, Practice and Progress tabs. Both interfaces build the same typed Request and use AgentController. Each GUI tab displays loading, success, empty and error states. AI calls run outside the Swing event-dispatch thread; observer updates return to that thread. CLI supports human-readable and JSON output.

ID	GUI	CLI command (proposed)
F01	Materials / Import	import –course C –file notes.pdf
F02	Materials / Summarise	summarise –course C –material M
F03	Materials / Concepts	concepts –course C –material M
F04	Ask / Question	ask –course C –question "..."
F05	Plan / Generate	plan –course C –exam YYYY-MM-DD –availability slots.json –strategy deadline
F06	Practice / Flashcards	cards –course C –topic T –count 10
F07	Practice / Quiz	quiz –course C –topic T –count 5
F08	Practice / Submit	grade –quiz Q –answers A,C,SKIP,D,A –attempt ID
F09	Progress / Topics	progress –course C
F10	Plan / Adapt / Confirm / Undo	adapt –plan P –strategy weak; confirm –proposal R; undo –plan P

CLI prerequisites: create/select a local workspace with course create/select. A study launcher dispatches commands. Exit codes: invalid input (2), external/model failure (3), storage/version conflict (4). Proposal IDs and undo snapshots are persisted, allowing confirmation or undo from a later CLI process. Dated availability is required for planning; a daily minutes setting alone is insufficient.

4. Feature specifications and detailed use cases

Each feature below has one primary use case. Actor: Student for all cases; the external AI service is a supporting actor for F02-F08 and F10. Every feature can be invoked through either interface. Numbered workflows are the main success scenarios. Alternative cases apply at the relevant step, and failed writes preserve the previous committed state.

F01 / UC01 - Import course materials

Goal / description: Import course materials. **Classification:** Deterministic.

GUI: Materials > Import. **CLI:** import –course C –file notes.pdf.

Input: Course ID and PDF/TXT file, up to 20 MB. **Output:** Indexed material with page/section references.

Preconditions: A course workspace is selected; local storage is writable. **Trigger:** The student selects a file and confirms import.

1. Validate type, size and course ID.
2. Extract text and split it into page-labelled chunks.
3. Save the material and chunks atomically; show import status.

Alternative / exception flows: Reject unsupported, encrypted or empty files. Scanned PDFs without text require the student to supply a text version; OCR is outside scope. Duplicate content is reported without another import.

Postconditions: The material and its page-labelled chunks are saved atomically and available for retrieval. Failed or duplicate imports create no partial material.

Related design: SD01. Classes: MaterialService; TextExtractor; Material; Chunk; StudyRepository. Patterns: Facade.

Feature realisation: MaterialService.importMaterial() calls TextExtractor.extract(); Material creates its Chunk objects and StudyRepository.saveMaterial() persists them. Retrieval uses only successfully committed chunks.

F02 / UC02 - Summarise selected materials

Goal / description: Summarise selected materials. **Classification:** Hybrid.

GUI: Materials > Select > Summarise. **CLI:** summarise –course C –material M.

Input: Material IDs and desired summary length. **Output:** Saved summary with source references.

Preconditions: At least one text material is indexed. **Trigger:** The student requests a summary.

1. Load selected chunks within the context limit.
2. Ask the model for a structured summary based only on those chunks.
3. Validate source IDs, save the result and display it.

Alternative / exception flows: Reject missing IDs. Process long selections in batches and combine summaries. If evidence is insufficient, say so. On invalid model output, retry once, then report failure without saving a result.

Postconditions: A validated summary with source references is saved and displayed. If generation or validation fails, existing artifacts remain unchanged.

Related design: SD02. Classes: LearningAgent; ToolRegistry; RetrievalTool; LLMClient; StudyRepository. Patterns: Facade, Adapter.

Feature realisation: AgentController.submit() delegates to LearningAgent.run(). The agent obtains chunks through ToolRegistry.execute("retrieve", args), calls LLMClient.generate(), validates references and uses StudyRepository.saveArtifact().

F03 / UC03 - Extract key concepts

Goal / description: Extract key concepts. **Classification:** Hybrid.

GUI: Materials > Extract concepts. **CLI:** concepts –course C –material M.

Input: Selected materials. **Output:** Editable concept list with definitions and source references.

Preconditions: Selected materials contain extracted text. **Trigger:** The student requests concepts.

1. Retrieve selected material chunks.
2. Generate concepts and definitions with source IDs.
3. Validate and deduplicate concept labels; save the concept artifact.

Alternative / exception flows: Show an empty-state explanation if no concepts can be supported. Reject unknown source IDs and malformed concept records; allow the student to correct labels before saving.

Postconditions: A validated, deduplicated concept list with definitions and source references is saved and displayed. Rejected or failed generation creates no artifact.

Related design: SD03. Classes: LearningAgent; ToolRegistry; RetrievalTool; LLMClient; StudyRepository. Patterns: Facade, Adapter.

Feature realisation: LearningAgent.run() orchestrates RetrievalTool through ToolRegistry, asks LLMClient.generate() for concept records and validates them before StudyRepository.saveArtifact(). Labels are deduplicated deterministically.

F04 / UC04 - Answer questions from course materials

Goal / description: Answer questions from course materials. **Classification:** Hybrid.

GUI: Ask > Enter question. **CLI:** ask –course C –question "What is polymorphism?".

Input: Question, course ID and optional prior conversation ID. **Output:** Answer with material/page references or an explicit insufficient-evidence response.

Preconditions: At least one material is indexed in the selected course. **Trigger:** The student submits a question.

1. Load bounded conversation context and identify a retrieval query.
2. Retrieve relevant chunks; allow a refined search within the agent step limit.
3. Generate a source-grounded answer, validate citations and save the exchange.

Alternative / exception flows: Ask for clarification if the course or question is ambiguous. If retrieval finds no relevant evidence, do not invent an answer. API failures leave prior conversation history unchanged.

Postconditions: A source-grounded answer and its conversation exchange are saved and displayed. Insufficient evidence is reported explicitly; API failure preserves prior history.

Related design: SD04. Classes: LearningAgent; ToolRegistry; RetrievalTool; LLMClient; StudyRepository. Patterns: Facade, Adapter.

Feature realisation: LearningAgent.run() loads context from StudyRepository.loadContext(), selects retrieve via ToolRegistry.execute(), and calls LLMClient.generate() with the evidence. Valid output is stored with saveArtifact(), including the conversation ID.

F05 / UC05 - Generate a personalised study plan

Goal / description: Generate a personalised study plan. **Classification:** Hybrid.

GUI: Plan > Generate. **CLI:** plan –course C –exam YYYY-MM-DD –availability slots.json –strategy deadline.

Input: Exam date, dated availability slots, selected topics and strategy. **Output:** Saved feasible StudyPlan with sessions and explanations.

Preconditions: Materials exist; exam date is in the future; availability is nonempty. **Trigger:** The student submits planning constraints.

1. Retrieve course topics and previous progress.
2. Use the model to propose topic priorities and estimated effort.
3. Use the selected scheduling strategy to allocate sessions; validate constraints, save and display the plan.

Alternative / exception flows: Reject past dates and negative durations. If work exceeds capacity, show unscheduled topics and request reduced scope or more time; never silently overlap sessions or exceed available time.

Postconditions: A valid study plan is saved and displayed. If constraints cannot be met, the conflict and unscheduled topics are reported without saving an invalid plan.

Related design: SD05. Classes: LearningAgent; StudyPlanner; SchedulingStrategy; StudyPlan; StudyRepository. Patterns: Facade, Adapter, Strategy.

Feature realisation: LearningAgent.run() retrieves material/progress tools and requests topic estimates from LLMClient. StudyPlanner.createPlan() uses SchedulingStrategy.allocate() and validate(). StudyRepository.savePlan() persists only a valid plan.

F06 / UC06 - Generate flashcards

Goal / description: Generate flashcards. **Classification:** Hybrid.

GUI: Practice > Flashcards > Generate. **CLI:** cards –course C –topic T –count 10.

Input: Course, topic and requested count (1-30). **Output:** Saved flashcard deck with prompts, answers and references.

Preconditions: Relevant course evidence is available. **Trigger:** The student requests a flashcard deck.

1. Retrieve topic evidence.
2. Generate distinct question-answer cards.
3. Validate count, nonempty fields and source IDs; save and display the deck.

Alternative / exception flows: Reject counts outside 1-30. If the material cannot support the count, return a clearly labelled smaller deck. A card without valid evidence is excluded and the reduction is reported.

Postconditions: A validated flashcard deck is saved and displayed, with any reduction in the requested count reported. Failed generation creates no incomplete deck.

Related design: SD06. Classes: LearningAgent; RetrievalTool; LLMClient; LearningArtifact; StudyRepository. Patterns: Facade, Adapter.

Feature realisation: LearningAgent.run() uses RetrievalTool and LLMClient.generate(). Each validated card is stored inside a LearningArtifact by StudyRepository.saveArtifact(); the GUI supports reveal-answer interaction without another AI call.

F07 / UC07 - Generate a multiple-choice quiz

Goal / description: Generate a multiple-choice quiz. **Classification:** Hybrid.

GUI: Practice > Quiz > Generate. **CLI:** quiz –course C –topic T –count 5.

Input: Course, topic, difficulty and count (1-20). **Output:** Saved Quiz with questions, options, answer keys and references.

Preconditions: Relevant course material is indexed. **Trigger:** The student requests a quiz.

1. Retrieve evidence and selected difficulty.
2. Generate questions with four choices, one answer key and cited explanations.
3. Validate question structure and answer-key bounds; save quiz and show questions without keys.

Alternative / exception flows: Reject invalid counts/difficulty. Reject duplicate questions, missing references or invalid keys. If generation still fails after one repair attempt, do not save an incomplete quiz.

Postconditions: A validated quiz, answer key and source references are saved; the student sees questions without the key. Invalid generation creates no incomplete quiz.

Related design: SD07. Classes: LearningAgent; LLMClient; Quiz; Question; StudyRepository. Patterns: Facade, Adapter.

Feature realisation: LearningAgent.run() retrieves evidence and invokes LLMClient.generate(). Validated question records become Quiz and Question objects. StudyRepository.saveQuiz() stores the key separately from the student-facing presentation.

F08 / UC08 - Grade quiz answers and explain mistakes

Goal / description: Grade quiz answers and explain mistakes. **Classification:** Hybrid.

GUI: Practice > Open quiz > Submit answers. CLI: grade --quiz Q --answers A,C,B,D,A --attempt ID.

Input: Quiz ID and one answer per question. **Output:** Deterministic score, per-question results and evidence-based feedback.

Preconditions: A saved quiz exists; submission answers refer to its questions. **Trigger:** The student submits the completed quiz.

1. Validate answer IDs and compare selections against the stored key.
2. Save the attempt and update per-topic progress exactly once.
3. Request explanations for wrong answers using stored evidence; display score even if explanations fail.

Alternative / exception flows: Reject malformed answers; unanswered questions score zero after confirmation. Repeated submission of the same attempt ID returns the existing result. An AI failure does not change the score or duplicate progress.

Postconditions: The attempt and deterministic score are committed once; progress reflects that attempt. Feedback is displayed if available. Feedback failure preserves the score; duplicate IDs add no observation.

Related design: SD08. Classes: QuizService; Quiz; StudyRepository; ProgressTracker; ProgressListener; LearningAgent. Patterns: Facade, Observer, Adapter.

Feature realisation: QuizService.grade() loads Quiz, evaluates the stored key and invokes StudyRepository.recordAttempt() atomically. ProgressTracker.record() notifies ProgressListener.update(). LearningAgent.explainMistakes() generates optional explanations; its output cannot modify the score.

Scoring rule. A confirmed skipped answer is a graded question with zero credit. Both quiz score and topic accuracy use correct / graded questions. Replaying an attempt ID never adds another observation. Each question has one primary topic.

F09 / UC09 - Identify weak topics and track progress

Goal / description: Identify weak topics and track progress. **Classification:** Deterministic.

GUI: Progress > Topic results. **CLI:** progress –course C.

Input: Course ID; stored quiz attempts. **Output:** Topic accuracy, evidence count and ranked weak-topic list.

Preconditions: A course workspace exists. **Trigger:** The student opens progress or completes a quiz.

1. Load committed attempts for the course.
2. Group answers by topic and calculate correct / graded counts (including confirmed skipped questions in the denominator).
3. Mark topics below 60% with at least three graded questions as weak; show insufficient-data topics separately.

Alternative / exception flows: No attempts produces an empty state, not a zero-ability judgement. Fewer than three questions is insufficient evidence. Corrupt storage reports an error instead of fabricated statistics.

Postconditions: Topic accuracy, evidence counts and weak topics are displayed, or an empty/insufficient-data state is shown. Viewing progress does not change stored attempts.

Related design: SD09. Classes: ProgressTracker; StudyRepository; ProgressSummary; ProgressListener. Patterns: Facade, Observer.

Feature realisation: AgentController.submit() calls ProgressTracker.analyse(). StudyRepository.loadAttempts() provides persisted attempts; the tracker calculates rates and returns ProgressSummary. ProgressListener.update() refreshes the dashboard after committed changes.

Evidence rule. The threshold of three refers to graded question observations across distinct committed attempts; confirmed skips count as incorrect observations. Repeated attempts are shown as practice history, not independent proof of mastery.

F10 / UC10 - Adapt and undo a study-plan revision

Goal / description: Adapt and undo a study-plan revision. **Classification:** Hybrid.

GUI: Plan > Adapt > Preview > Confirm; Undo. **CLI:** adapt –plan P –strategy weak; confirm –proposal R; undo –plan P.

Input: Current plan/version, completed sessions, progress, new availability. **Output:** Preview then saved revised plan, or restoration of previous plan.

Preconditions: A saved plan exists; changed constraints are valid. **Trigger:** The student requests adaptation or undo.

1. Load current plan and topic progress; preserve completed sessions.
2. Generate priorities and create a valid candidate with StudyPlanner.
3. Show a preview; on confirmation execute RevisePlanCommand with an expected version. Support one-level undo if the saved version has not changed.

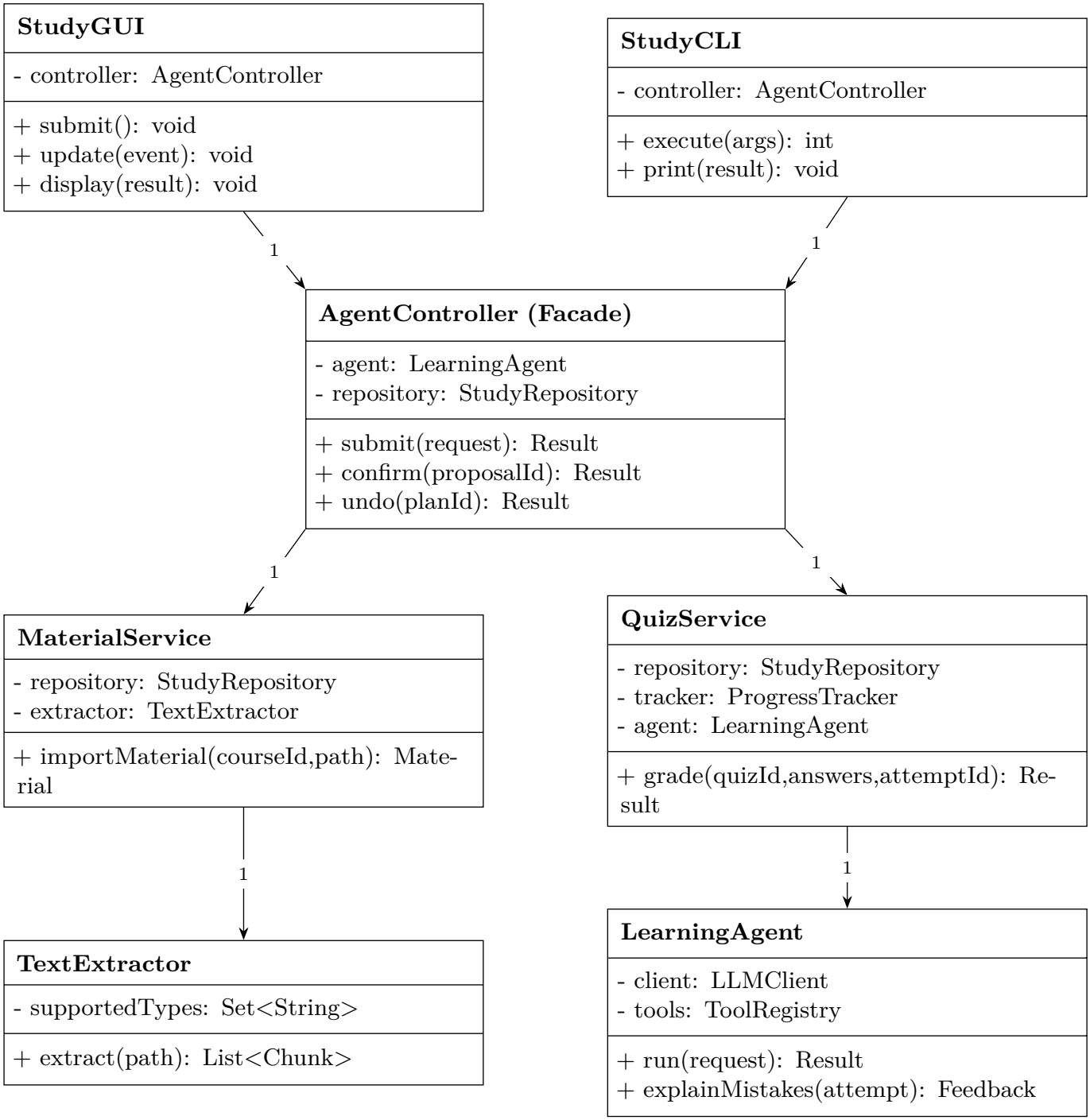
Alternative / exception flows: A rejected preview changes nothing. Capacity conflict requires user changes. A version conflict aborts confirmation or undo and reloads the latest plan. Undo does not erase quiz attempts or completed-study history.

Postconditions: A preview alone leaves the current plan unchanged. Confirmation saves a valid revised plan and undo snapshot; successful undo restores the prior schedule as a new version. Conflicts cause no write.

Related design: SD10a and SD10b. Classes: AgentController; LearningAgent; StudyPlanner; PlanCommand; RevisePlanCommand; StudyRepository. Patterns: Facade, Adapter, Strategy, Command.

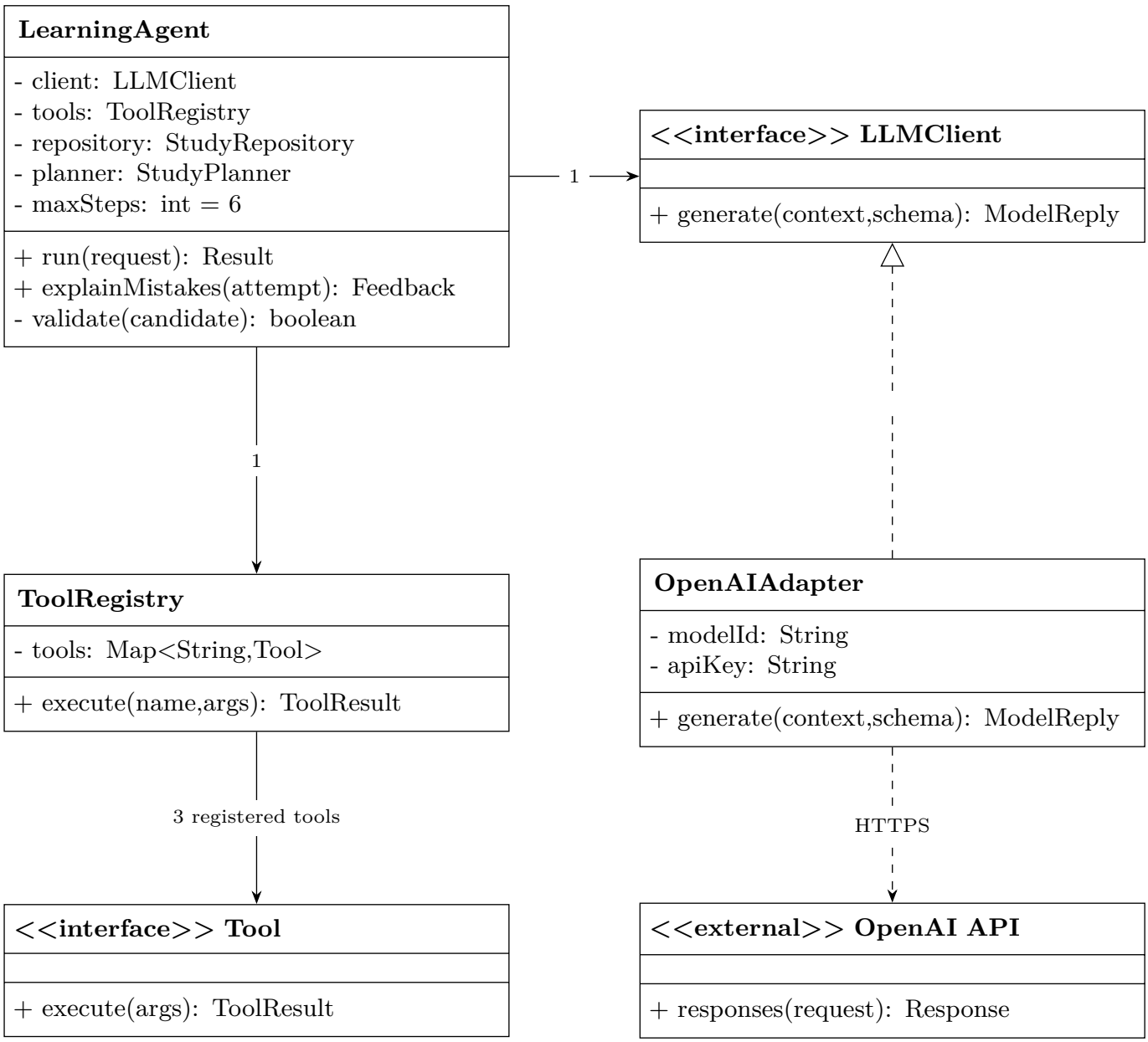
Feature realisation: LearningAgent.run() proposes a candidate via StudyPlanner.createPlan(). AgentController stores a pending RevisePlanCommand; confirm() calls execute(), which saves through StudyRepository.savePlan(). undo() calls the command to restore its before snapshot using version checking.

5.1 Class model: presentation and application

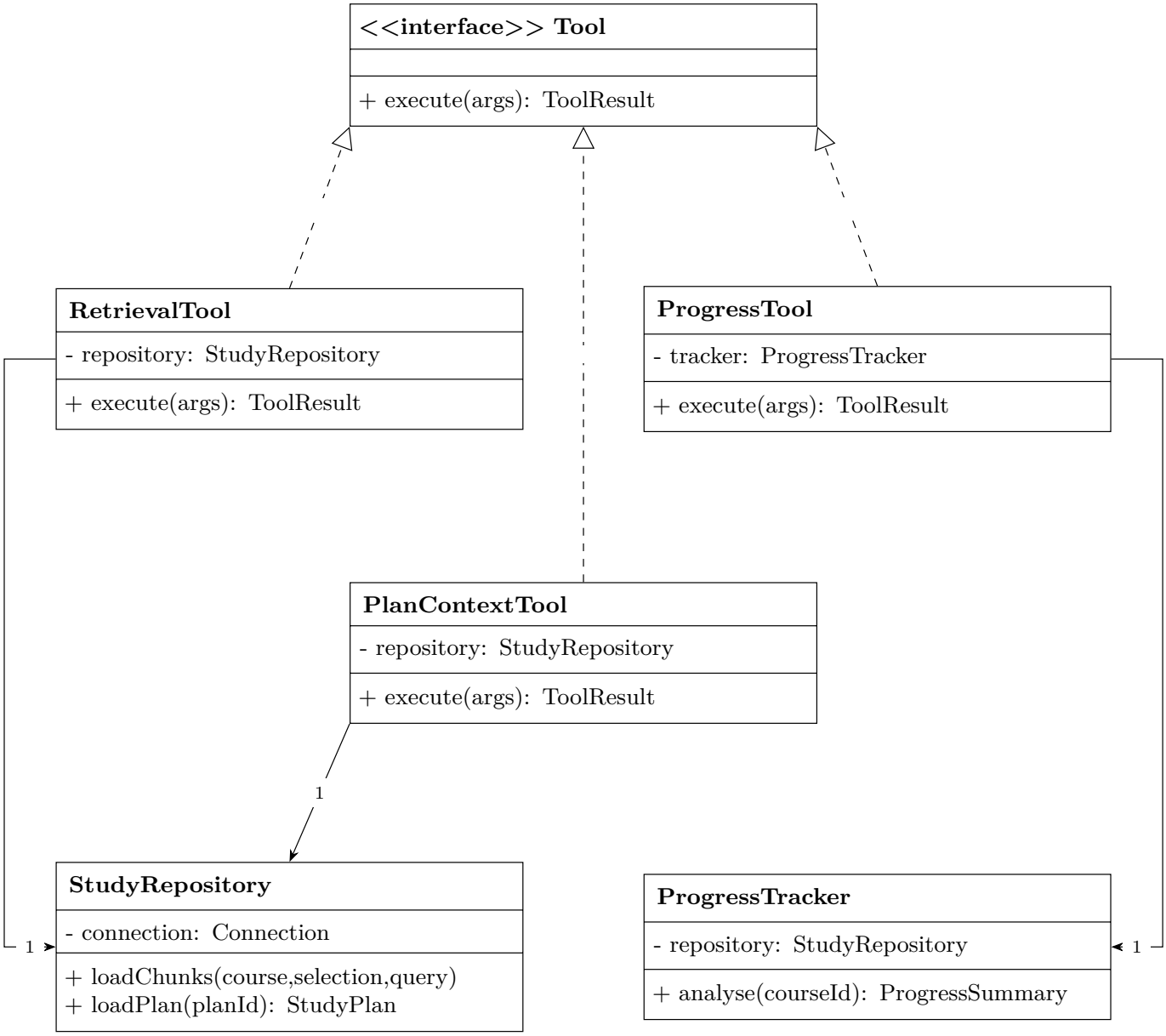


The controller also holds one ProgressTracker and one LearningAgent; those cross-view associations are shown in the relationship table. GUI and CLI depend on the controller.

5.2 Class model: agent and model adapter

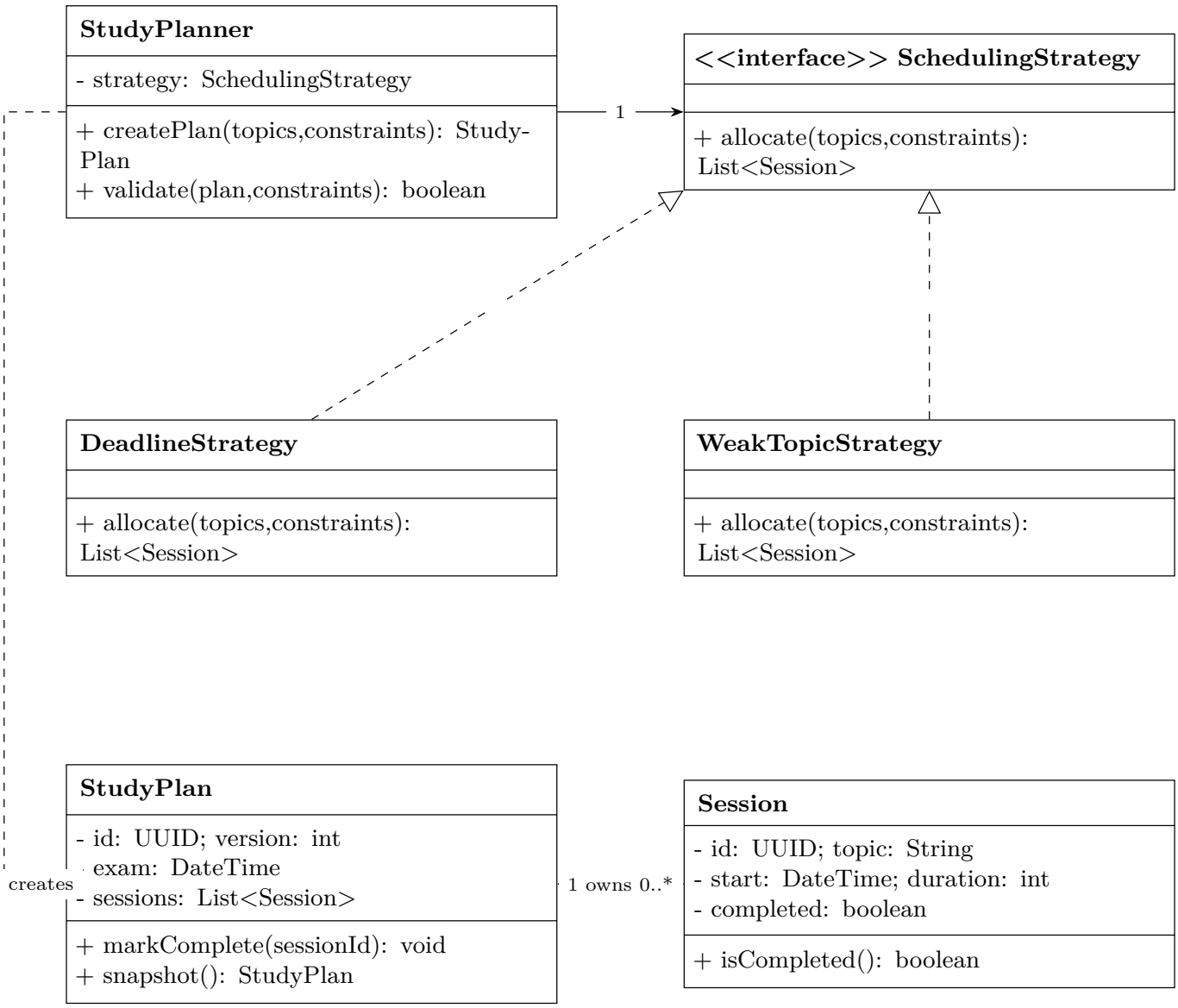


5.3 Class model: separate tool implementations

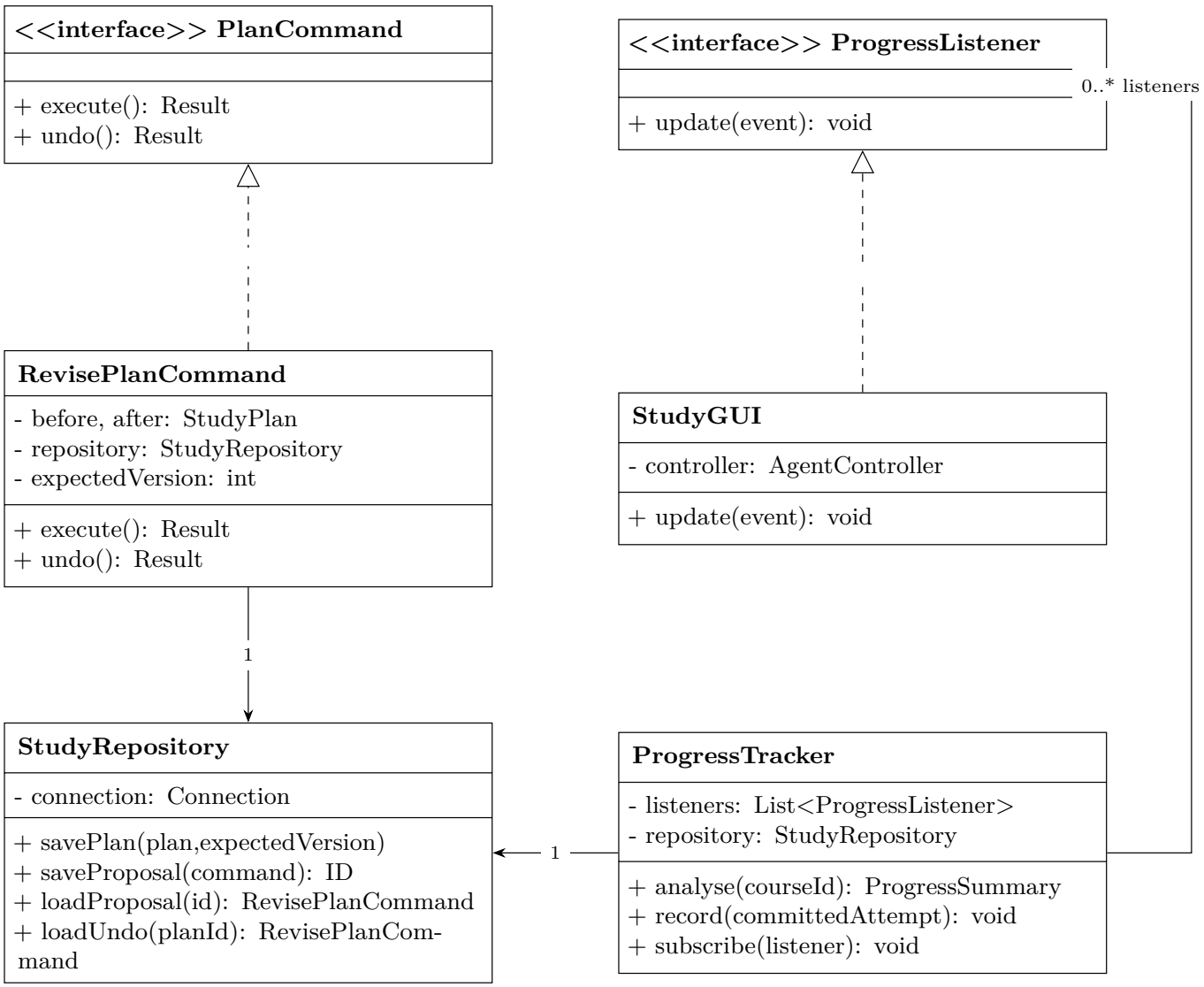


Each concrete tool implements the Tool contract. RetrievalTool ranks course-scoped chunks; ProgressTool computes topic results; PlanContextTool reads the current plan. All tools are read-only.

5.4 Class model: scheduling Strategy

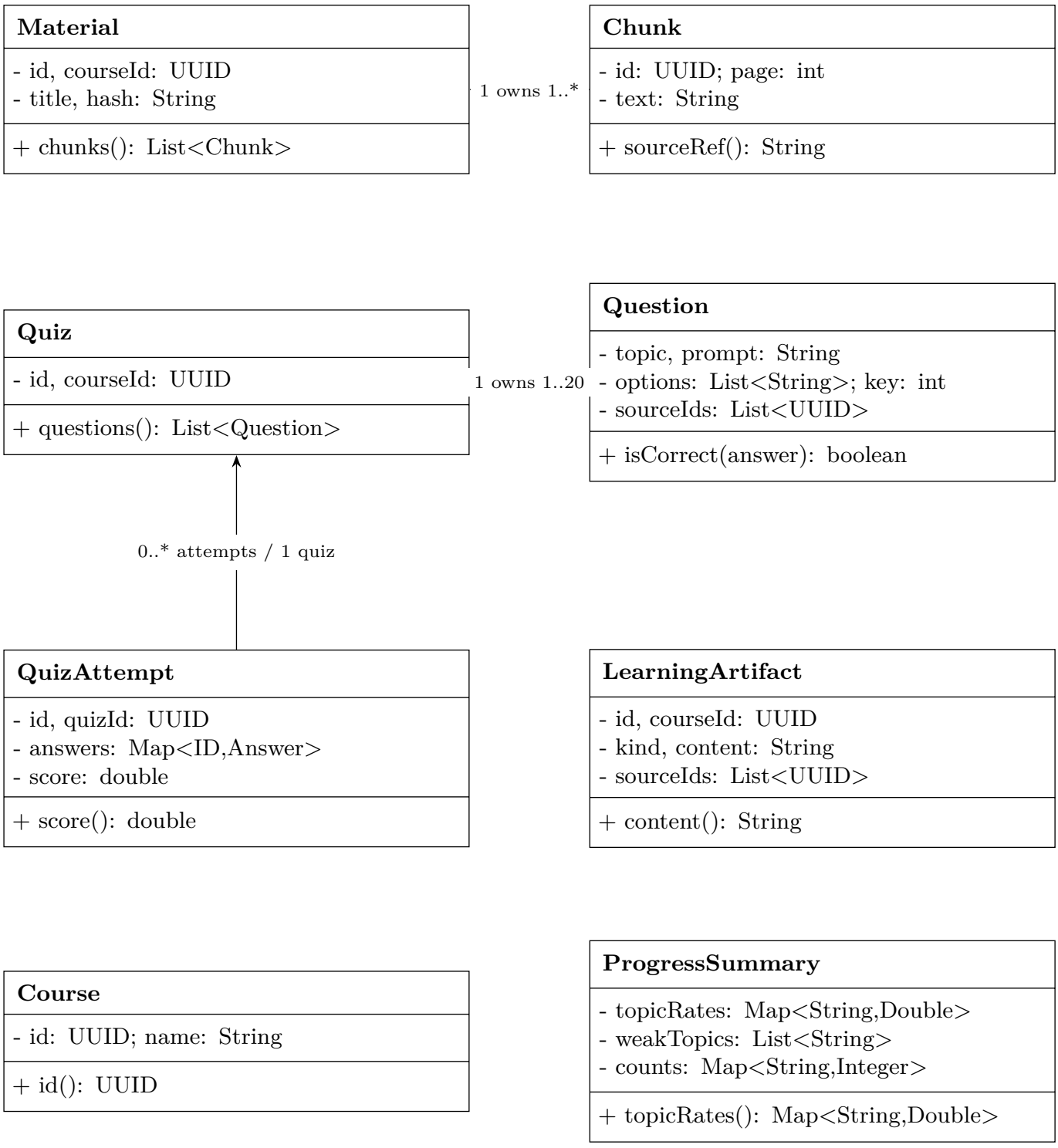


5.5 Class model: Command and Observer



AgentController invokes PlanCommand. RevisePlanCommand owns independent before/after plan snapshots. ProgressTracker notifies subscribed GUI observers only after a successful new attempt commit. record() publishes an event; it does not insert a second attempt.

5.6 Class model: domain records



5.7 Cross-view relationships and repository contract

Relationship	Meaning
StudyGUI / StudyCLI -> AgentController	One controller per boundary; the controller has no concrete UI dependency.
AgentController -> services	One MaterialService, QuizService, ProgressTracker, LearningAgent and StudyRepository.
LearningAgent -> dependencies	One LLMClient, ToolRegistry, StudyPlanner and StudyRepository.
Course -> domain records	One course associates with 0..* Material, Quiz, StudyPlan and LearningArtifact records.
RevisePlanCommand -> StudyPlan	Two independent snapshots: before and after. One repository receiver.
ToolRegistry -> Tool	Three registered tools initially; Tool instances are referenced, not owned by composition.

Notation. + public; - private. Solid arrows are navigable associations. Dashed arrows with open triangles are interface realization; dashed ordinary arrows are dependencies. Filled diamonds mark composition at the owner. Repeated class names in different views denote the same class. No domain subclassing is required.

StudyRepository methods

saveMaterial(material): Material; loadChunks(course,selection,query): List<Chunk>

loadContext(course): Context; saveArtifact(artifact): LearningArtifact

saveQuiz(quiz): Quiz; loadQuiz(id): Quiz; loadAttempts(course): List<QuizAttempt>

recordAttempt(attempt): CommitResult (stored attempt plus isNew flag)

savePlan(plan,expectedVersion): StudyPlan; loadPlan(id): StudyPlan

saveProposal(command): ID; loadProposal(id): RevisePlanCommand

loadUndo(planId): RevisePlanCommand

Plan writes atomically persist the new version and undo snapshot. Confirmation loads the persisted proposal and verifies its expected version. Undo loads the persisted command history, restores the previous schedule as a new version, and fails if any intervening plan/completion update occurred. Material imports and attempts are transactional. Attempt IDs are unique; retries return the stored attempt.

Supporting value records

Request(feature,courseId,inputs); Result(status,data,errors); ModelReply(toolCall or finalPayload); ToolResult(data,sourceIds); Constraints(exam,availability,timezone); ProgressEvent(courseId,attemptId); CommitResult(attempt,isNew). These immutable values validate required fields at construction. Repository failures use typed storage/conflict errors.

6. Five design patterns and rationale

Facade

AgentController is the facade; MaterialService, QuizService, ProgressTracker and LearningAgent are subsystem services; StudyGUI/StudyCLI are clients. Both entry points need the same application operations and error contracts. submit(), confirm() and undo() expose that boundary. Without the facade, both interfaces would duplicate dispatch, validation and service-coordination logic. Domain algorithms remain in their own services, avoiding a god controller.

Adapter

LLMClient is the target; OpenAIAdapter is the adapter; the external OpenAI API is the adaptee; LearningAgent is the client. The provider request/response format differs from the application ModelReply contract. The adapter translates requests, tool schemas, responses and errors. Without it, provider-specific formats would spread into planning and learning logic. Model configuration is replaceable without changing those callers.

Strategy

StudyPlanner is the context; SchedulingStrategy is the strategy interface; DeadlineStrategy and WeakTopicStrategy are concrete strategies. DeadlineStrategy allocates topics in syllabus order within earliest available slots. WeakTopicStrategy orders topics by ascending measured accuracy, retaining syllabus order for ties and placing insufficient-data topics next. Both obey the same schedule constraints. Without interchangeable strategies, StudyPlanner would grow conditional scheduling branches. The common contract allows adding a strategy without rewriting callers.

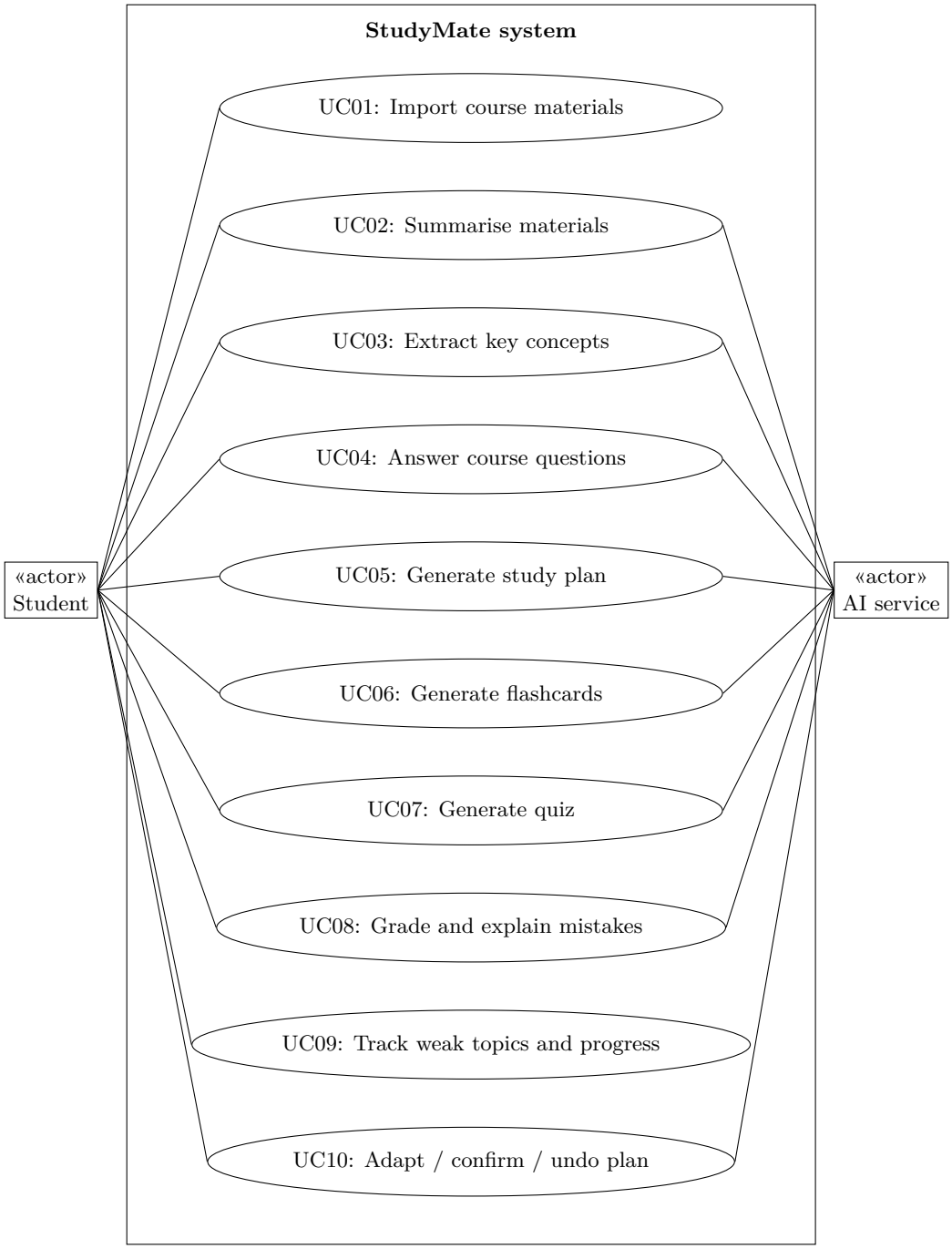
Observer

ProgressTracker is the subject; ProgressListener is the observer interface; StudyGUI is the concrete observer. A committed quiz attempt changes progress. record() publishes a course-scoped event, and update() refreshes the displayed summary. CLI reads results directly and does not require a persistent subscription. Without it, QuizService would need to know GUI widgets or the GUI would poll continuously. The tracker never notifies before a successful transaction.

Command

PlanCommand is the command interface; RevisePlanCommand is the concrete command; AgentController is the invoker; StudyRepository is the receiver; before/after StudyPlan snapshots are command state. A proposed plan revision needs delayed confirmation and one-level undo. execute() commits the candidate, while undo() restores the previous snapshot if the version is still current. Without a command object, GUI and CLI would duplicate pending-edit, snapshot and undo logic. Persisted proposals also allow confirmation in a separate CLI process.

7. UML use-case diagram

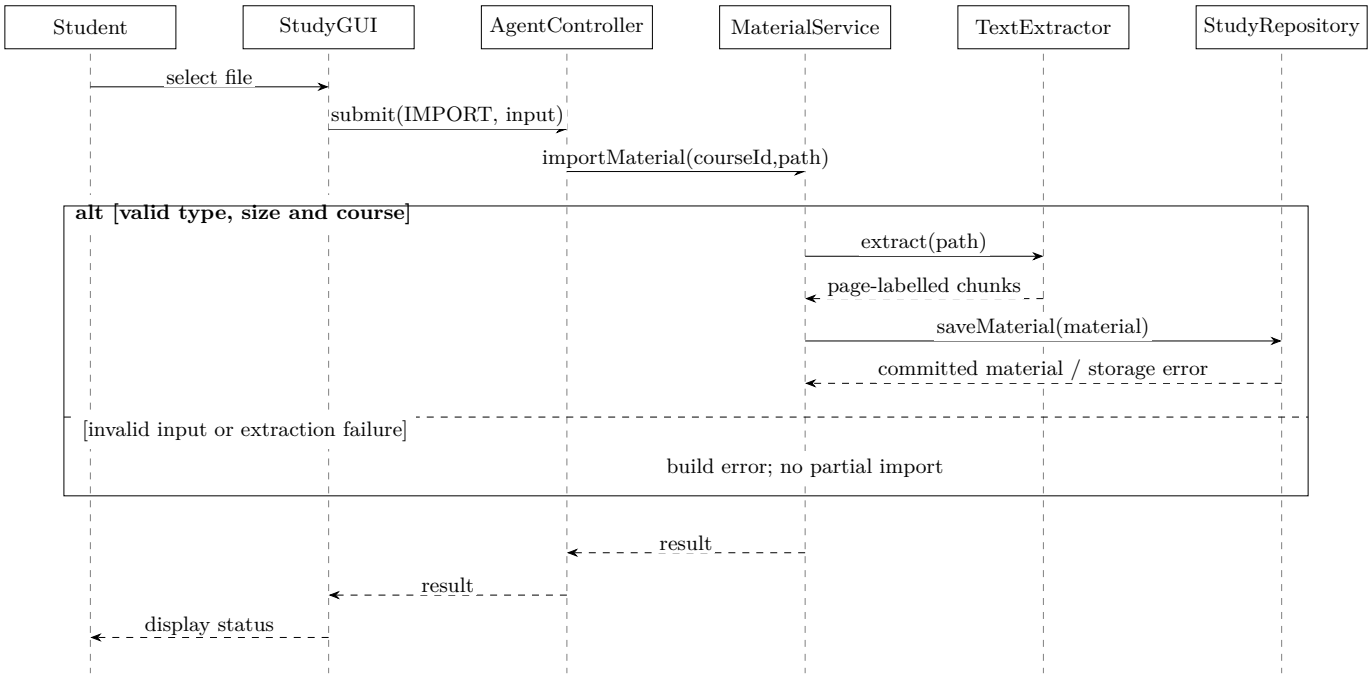


The Student is the primary actor for every use case. The AI service supports UC02–UC08 and UC10; UC08 uses it only for optional feedback. Deterministic grading, confirmation and undo do not require a model call. Section 4 supplies the full descriptions.

8. UML sequence diagrams

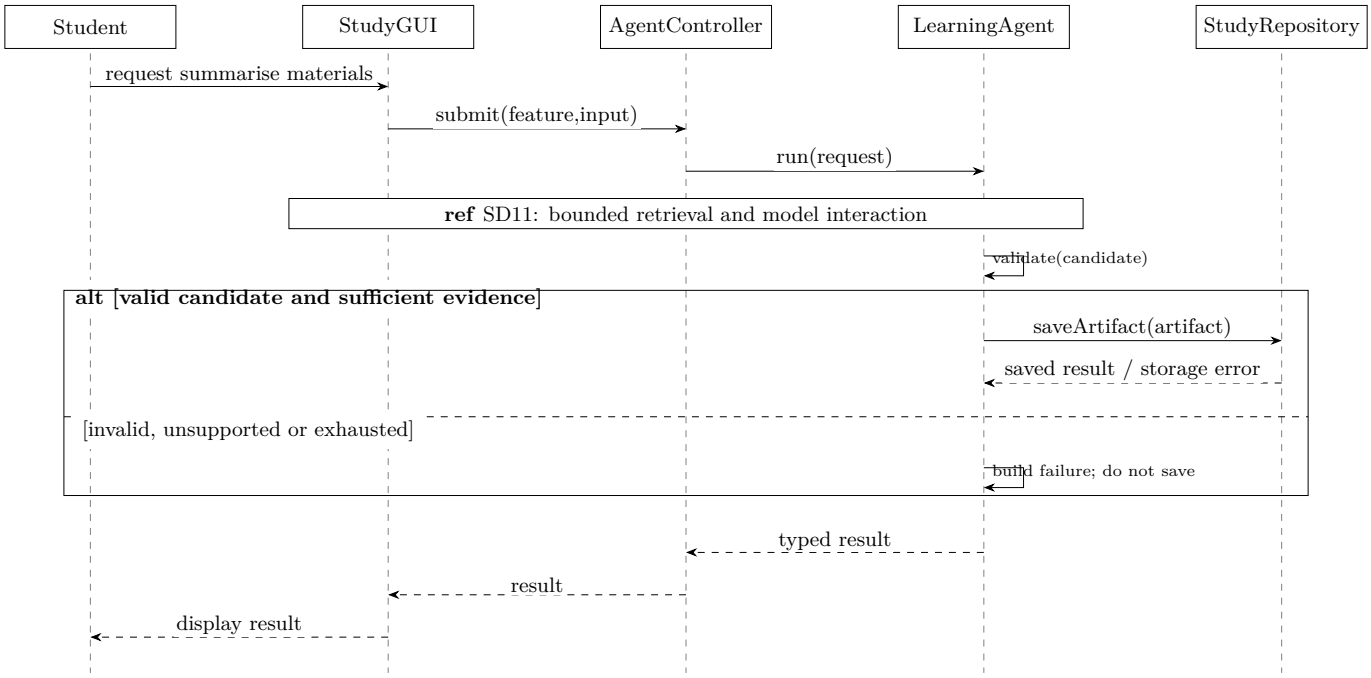
Each lifeline is an instance of the named class. GUI is the representative boundary; CLI submits the same controller requests. Solid arrows are calls; dashed arrows are returns. Combined fragments show guards and repetition. SD11 expands model/tool collaboration; feature diagrams use a ref frame for this shared interaction.

SD01 - Import course materials



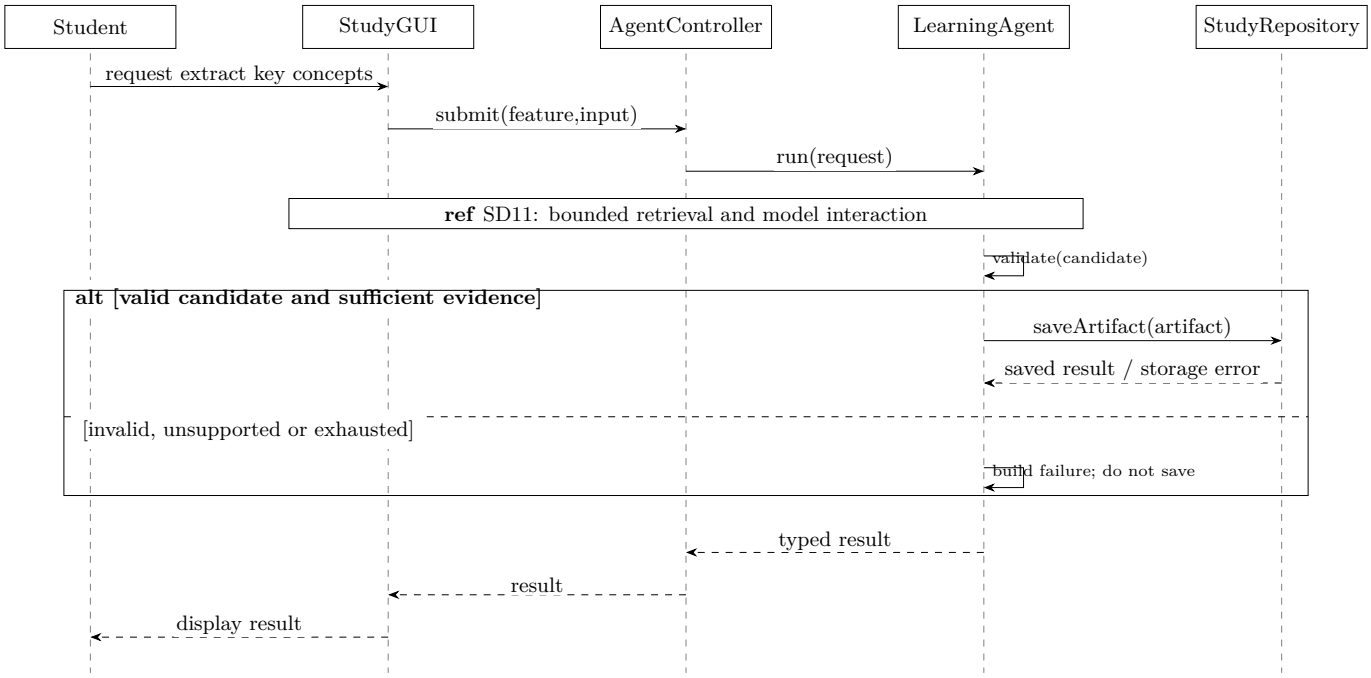
saveMaterial rejects duplicate hashes and commits material/chunks atomically. A failed extraction never reaches saveMaterial. The final returns merge the selected branch.

SD02 - Summarise materials



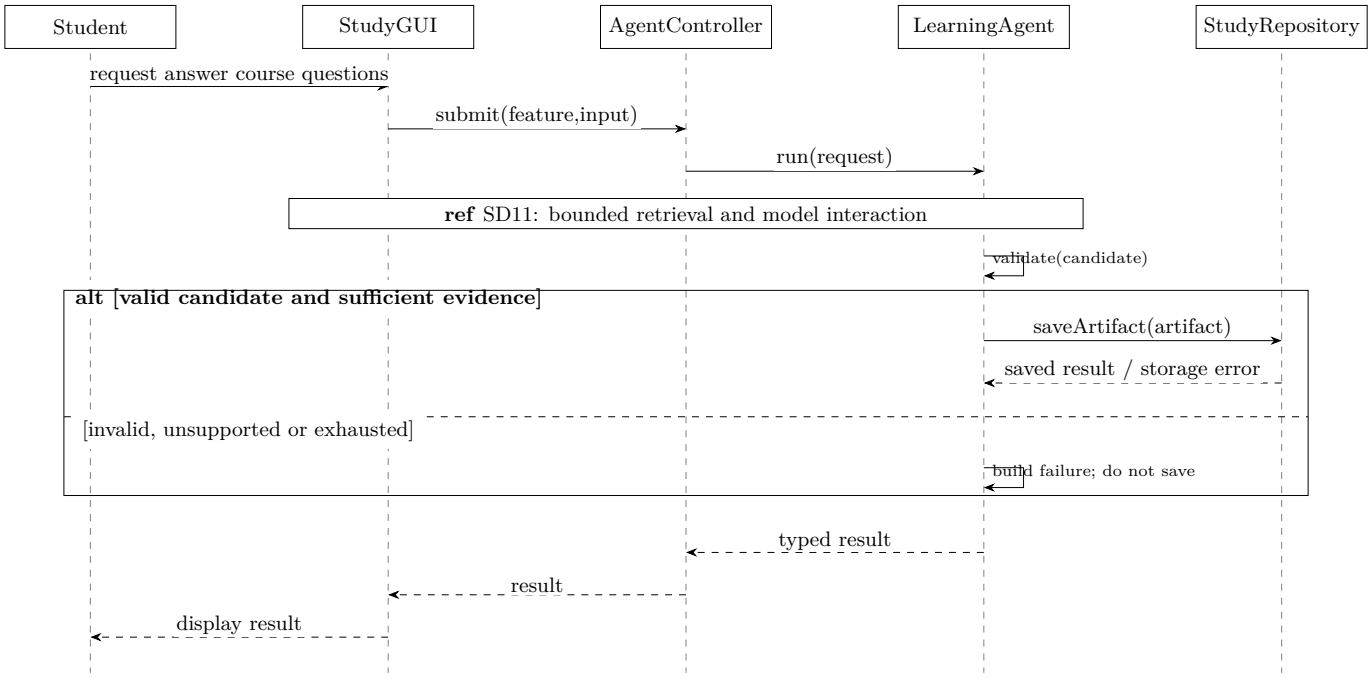
UC02 provides feature-specific validation and error cases. No invalid candidate is committed.

SD03 - Extract key concepts



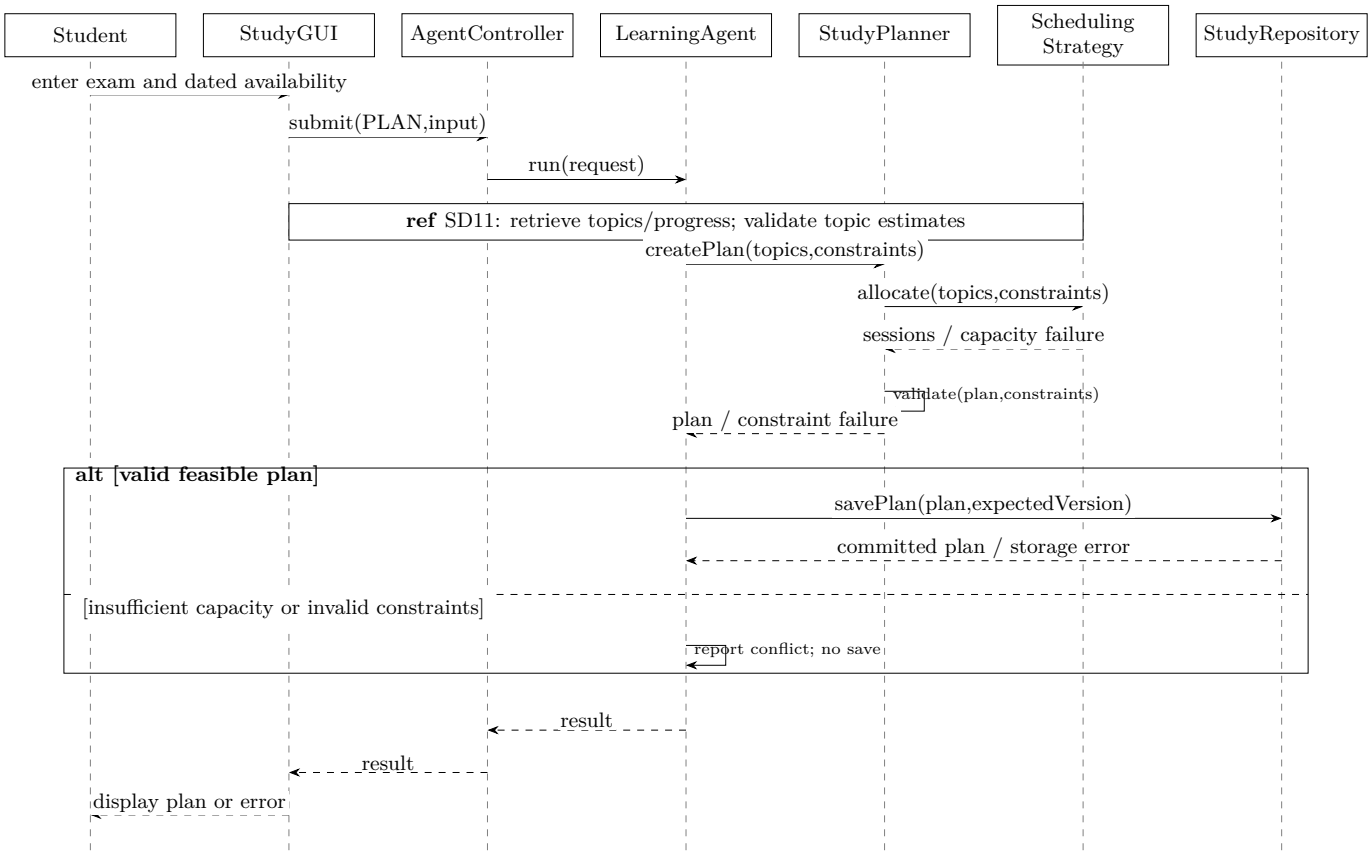
UC03 provides feature-specific validation and error cases. The editable concept result can be corrected in the GUI and revalidated before saving; CLI accepts equivalent edited fields.

SD04 - Answer course questions



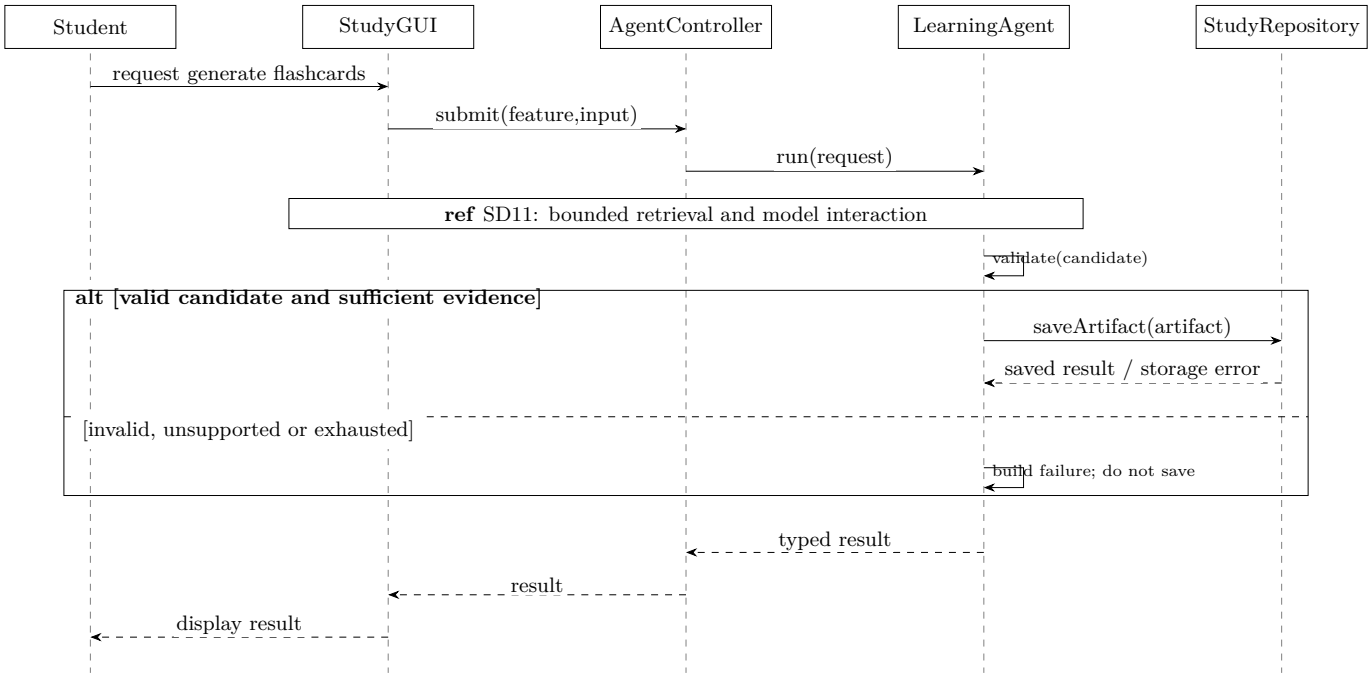
UC04 provides feature-specific validation and error cases. No invalid candidate is committed.

SD05 - Generate a personalised study plan



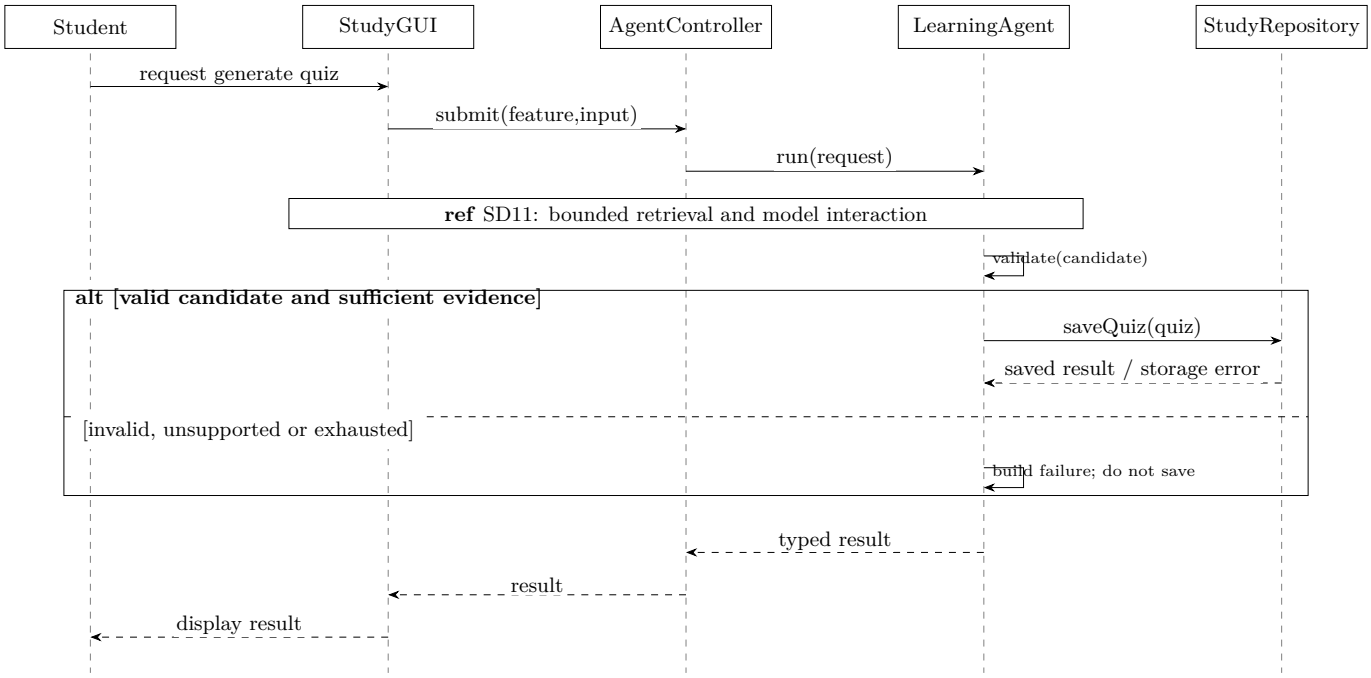
StudyPlanner enforces non-overlap, positive durations, dated availability and the exam deadline. Neither strategy may silently discard excess work; unscheduled topics are reported.

SD06 - Generate flashcards



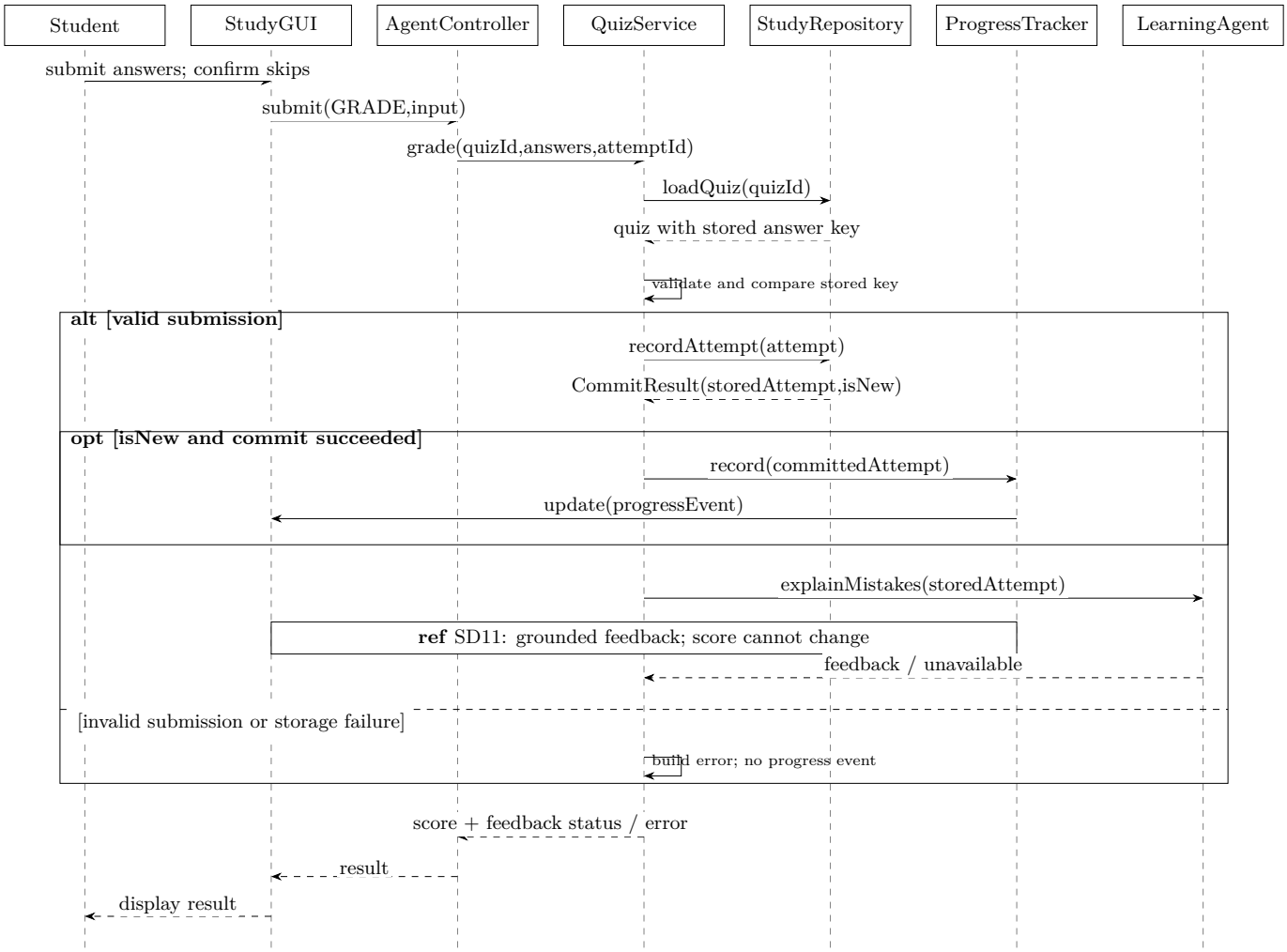
UC06 provides feature-specific validation and error cases. No invalid candidate is committed.

SD07 - Generate quiz



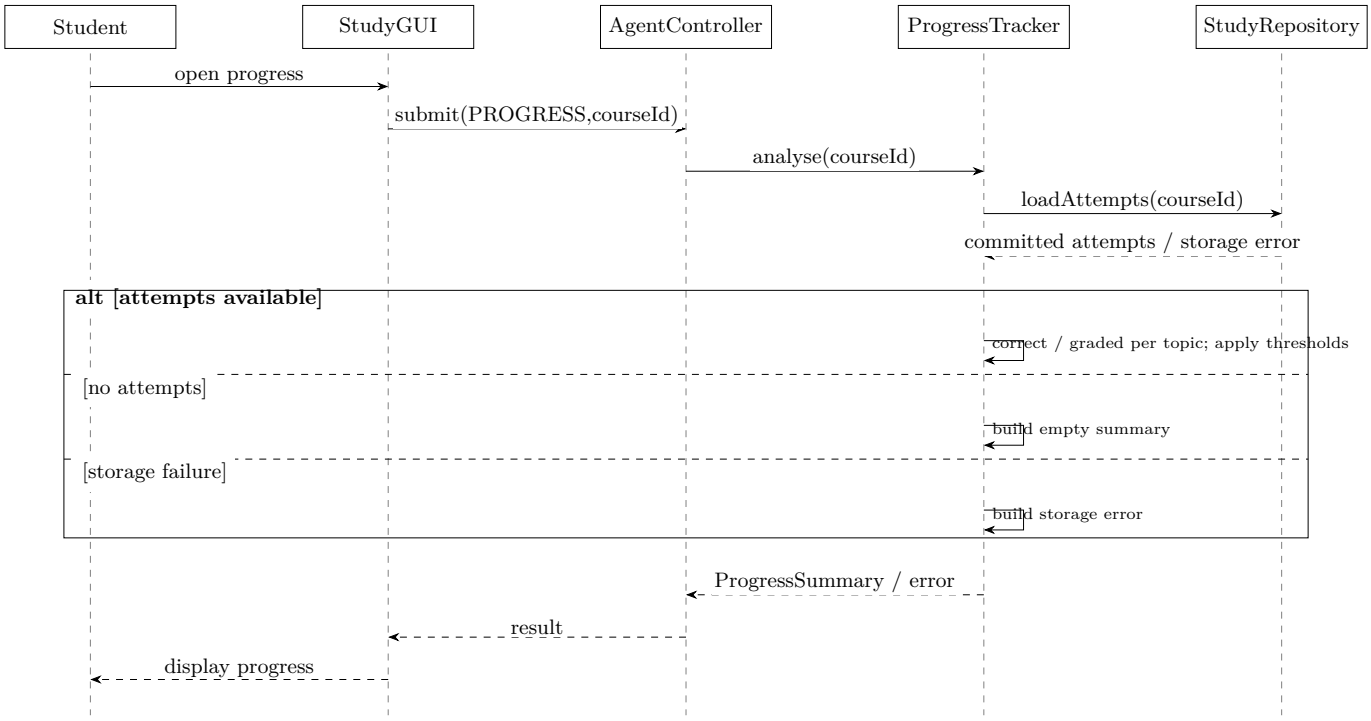
UC07 provides feature-specific validation and error cases. No invalid candidate is committed.

SD08 - Grade quiz and explain mistakes



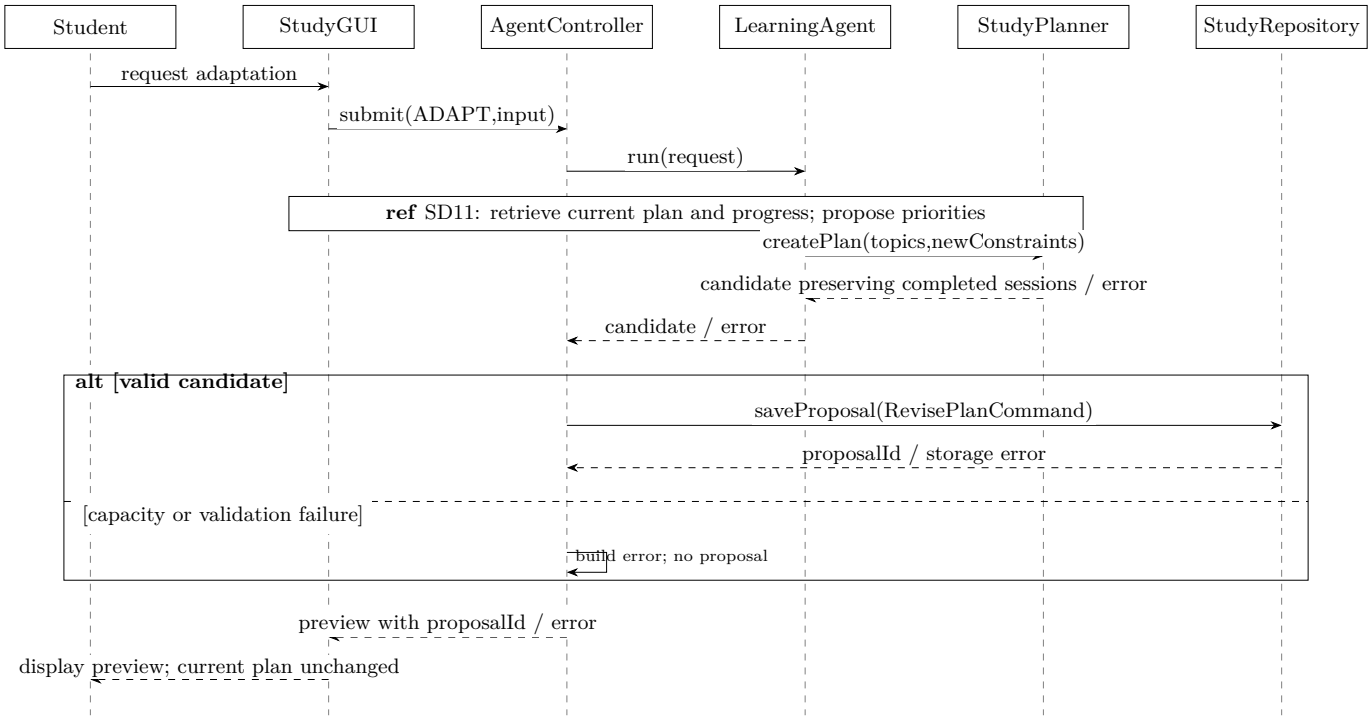
A duplicate attempt ID returns the original stored score and does not trigger another progress update. A storage error exits the success operand before feedback. Feedback failure preserves a successfully committed score. GUI update is dispatched onto the Swing event thread.

SD09 - Identify weak topics and track progress



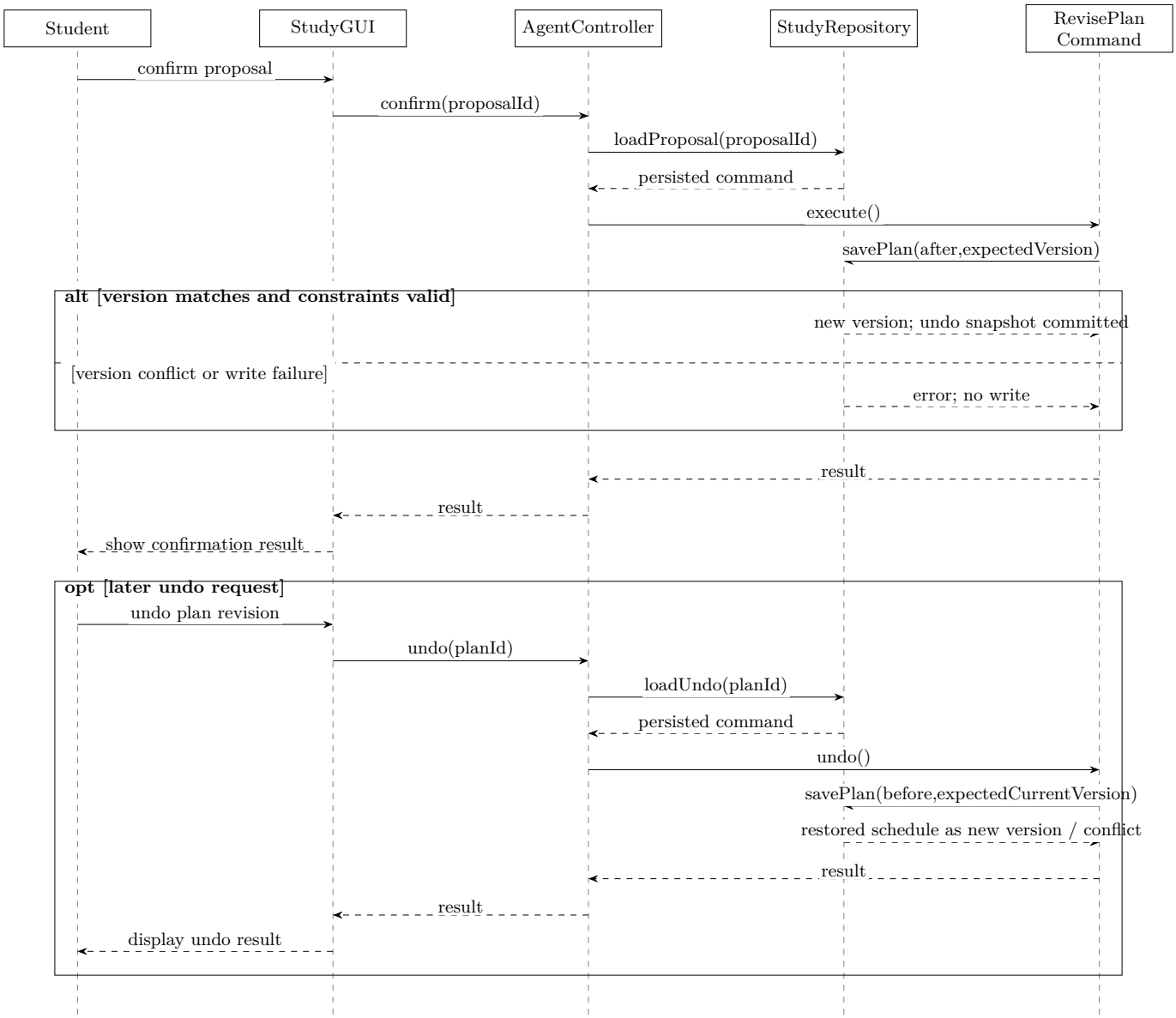
Confirmed skipped answers count as incorrect graded observations. Weak means below 60% with at least three graded observations. Less evidence is labelled insufficient data.

SD10a - Preview a study-plan revision



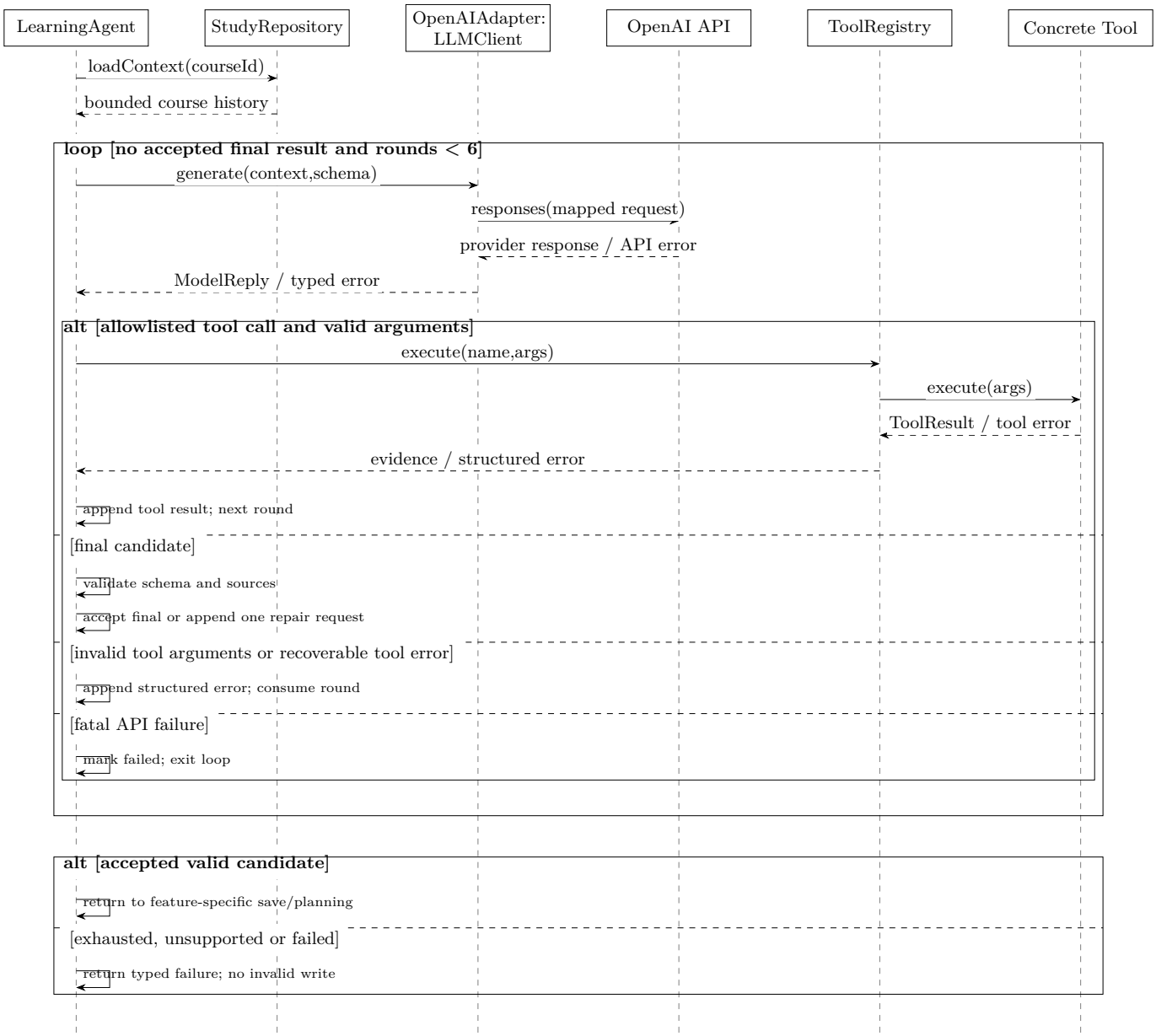
The proposal stores before/after snapshots and the expected current version. Model tools do not write plans. StudyPlanner applies the Strategy interaction shown in SD05.

SD10b - Confirm and undo a persisted revision



A rejected preview leaves the plan unchanged. One-level undo is permitted only while the revised plan is still current; intervening completion changes increment the version and block stale undo. Quiz attempts are never reverted.

SD11 - Bounded agent tool loop



Concrete Tool means one registered RetrievalTool, ProgressTool or PlanContextTool, never all three at once. Their collaborators appear in class view 5.3. Every generate() decision consumes one round; at most one malformed-output repair is allowed within six rounds. An accepted final reply or fatal failure ends the loop. A provider transport retry is limited to one transient retry per call, with a planned 30-second timeout. Insufficient evidence may prompt a refined search within the same budget.

9. Feature-to-design traceability

StudyGUI / StudyCLI and AgentController.submit() are common to every feature. Full feature realisations and use-case descriptions are in Section 4. SD11 expands LearningAgent generation, including optional feedback in F08.

Feature / type / UC / SD	Classes	Key methods	Patterns
F01 Import course materials; Deterministic; UC01; SD01	MaterialService, TextExtractor, StudyRepository	importMaterial; extract; saveMaterial	Facade
F02 Summarise materials; Hybrid; UC02; SD02; SD11	LearningAgent, RetrievalTool, LLMClient, StudyRepository	run; execute; generate; saveArtifact	Facade, Adapter
F03 Extract key concepts; Hybrid; UC03; SD03; SD11	LearningAgent, RetrievalTool, LLMClient, StudyRepository	run; execute; generate; saveArtifact	Facade, Adapter
F04 Answer course questions; Hybrid; UC04; SD04; SD11	LearningAgent, ToolRegistry, LLMClient, StudyRepository	run; loadContext; execute; generate; saveArtifact	Facade, Adapter
F05 Generate study plan; Hybrid; UC05; SD05; SD11	LearningAgent, StudyPlanner, SchedulingStrategy, StudyRepository	run; createPlan; allocate; validate; savePlan	Facade, Adapter, Strategy
F06 Generate flashcards; Hybrid; UC06; SD06; SD11	LearningAgent, RetrievalTool, LLMClient, LearningArtifact	run; execute; generate; saveArtifact	Facade, Adapter
F07 Generate quiz; Hybrid; UC07; SD07; SD11	LearningAgent, LLMClient, Quiz, Question, StudyRepository	run; generate; validate; saveQuiz	Facade, Adapter
F08 Grade and explain mistakes; Hybrid; UC08; SD08; SD11	QuizService, StudyRepository, ProgressTracker, LearningAgent	grade; recordAttempt; record; update; explainMistakes	Facade, Observer, Adapter
F09 Track weak topics and progress; Deterministic; UC09; SD09	ProgressTracker, StudyRepository, ProgressSummary	analyse; loadAttempts; update	Facade, Observer
F10 Adapt / confirm / undo plan; Hybrid; UC10; SD10a/b; SD11	AgentController, LearningAgent, StudyPlanner, RevisePlanCommand, StudyRepository	run; createPlan; saveProposal; loadProposal; confirm; execute; loadUndo; undo; savePlan	Facade, Adapter, Strategy, Command

10. Design quality, next stages and submission

OOP and SOLID decisions

Single responsibility: extraction, grading, scheduling, UI and model communication are separated. Open/closed: new Tool and SchedulingStrategy implementations extend behaviour through interfaces. Liskov substitution: every strategy returns sessions satisfying the same constraints or an explicit capacity failure. Interface segregation: LLMClient, Tool and ProgressListener expose small focused contracts. Dependency inversion: LearningAgent receives LLMClient rather than constructing OpenAIAdapter. Dependencies are supplied at application startup; no Singleton is required.

Planned validation in later stages

Stage 2 will implement the Java design with AI-assisted development and record representative prompts, human review and design changes. Stage 3 will use JUnit for extraction validation, source-ID checks, quiz scoring, idempotent attempts, scheduling constraints, observer notifications and command undo/version conflicts. Agent evaluation with KUMA will cover correct tool selection, use of retrieved evidence, clarification, invalid tool calls, unsupported questions, prompt injection in material text and bounded failure recovery. These are planned tests, not reported results.

Repository submission checklist

Place this report and editable diagram/report sources in one public EECS3311 GitHub repository. Suggested paths: docs/stage1/Stage1_StudyMate.pdf and docs/stage1/source/. Add a README explaining the proposed system and linking to the report. Do not upload API keys or copyrighted course slides to the public repository. Commit and push the report before the course deadline, verify access to the repository URL, and submit that URL through the course system. Continue in the same repository for Stages 2 and 3.

References and preparation note

[1] EECS3311 Fall 2026 Course Project Stage-1 Instruction, supplied stage1 (1)(1).pdf. Source of minimum requirements, nine deliverables and public-repository submission rules. [2] EECS3311 Software Design, Course Project: Design, Build, and Test an AI Agent, supplied Project-Instruction (1)(1).pdf. Source of three-stage scope and testing expectations. [3] Supplied course slides: OOP Design; UML; UML-II; SOLID; Creational, Structural and Behavioral Patterns. Used for course-aligned design terminology and notation. JUnit slides inform the later-stage testing plan. [4] OpenAI, GPT-4.1 mini model documentation: https://developers.openai.com/api/docs/models/gpt-4.1-mini (planned integration reference). The selected model remains a planned integration, not a tested implementation. Prepared with ChatGPT assistance. The student should review and understand the feature scope, UML relationships and pattern roles before submission. Project repository: https://github.com/ElenaBahrami/EECS3311-StudyMate.